

STM Creator Developer Manual

This manual is for developers maintaining STM Creator. It explains the repository layout, technology stack, architecture, local setup, frontend and backend module boundaries, database design, authentication, collaboration, testing, build, deployment, and common development tasks.

Production deployment:

- Frontend editor: <https://stm-8nizc.ondigitalocean.app/editor>
- Backend API used by the frontend source: <https://hammerhead-app-t8l9y.ondigitalocean.app>

1. Repository Structure

The workspace contains two main applications:

```
COMP8715/  
  stm_creator-frontend/  
  tern_backend/
```

High-level ownership:

- `stm_creator-frontend`: React/Vite client application.
- `tern_backend`: Express/TypeScript backend, PostgreSQL migrations, tests, Postman collection, Docker configuration.

The root also contains generated project documentation and existing PDF outputs. The root itself is not the main application package.

2. Technology Stack

Frontend

Area	Technology
Build tool	Vite
UI runtime	React
Language	TypeScript
Graph editor	React Flow / @xyflow/react
Styling	CSS, CSS Modules for admin dashboard
Realtime	Socket.IO client
Auth storage	Browser storage through the frontend auth helper
Import/export	EKS-style JSON helpers and PNG export helpers

Backend

Area	Technology
Runtime	Node.js
Web framework	Express
Language	TypeScript
Database	PostgreSQL with PostGIS
Realtime	Socket.IO
Auth	JWT, bcrypt, password pepper
Validation	Zod
Migrations	node-pg-migrate
Tests	Jest, Supertest, Socket.IO client tests, Newman/Postman
API docs	Swagger/OpenAPI setup
Deployment	Dockerfile and DigitalOcean-compatible Node service

3. High-Level Architecture

The system is a client-server application:

```
Browser
|
| HTTPS REST + Socket.IO
v
Frontend deployment
|
| VITE_API_BASE_URL
v
Backend API
|
| pg Pool
v
PostgreSQL / PostGIS
```

Frontend responsibilities:

- Route users through home, auth, editor, verification, admin, forbidden, and not-found pages.
- Hold graph editor state in React hooks.
- Convert backend model data into React Flow nodes and edges.
- Convert edited graph state into backend save payloads.
- Provide user interaction for states, transitions, comments, milestones, sharing, locks, and imports/exports.

- Connect to Socket.IO for collaboration.

Backend responsibilities:

- Authenticate and verify users.
- Enforce global and model-level permissions.
- Validate requests.
- Persist models, states, transitions, drivers, permissions, comments, locks, milestones, audit logs, and sessions.
- Serve REST APIs.
- Serve Socket.IO collaboration events.
- Run database migrations.

4. Local Development Environment

4.1 Backend Setup

From the backend directory:

```
cd tern_backend  
npm install
```

Create `.env` from `.env.example` :

```
NODE_ENV=development  
PORT=3000  
DATABASE_URL=postgres://postgres:postgres@localhost:5432/tern  
JWT_SECRET=replace-with-a-long-random-secret  
PASSWORD_PEPPER=replace-with-a-long-random-secret  
FRONTEND_URL=http://localhost:5173
```

Run migrations:

```
npm run migrate:up
```

Start the development server:

```
npm run dev
```

Health checks:

```
GET http://localhost:3000/auth/health  
GET http://localhost:3000/models/health
```

4.2 Backend Docker Setup

The backend includes Docker and compose configuration.

Common workflow:

```
cd tern_backend  
docker compose up --build
```

Use Docker when you want a reproducible database and app environment. Use direct Node execution when you want faster backend iteration.

4.3 Frontend Setup

From the frontend directory:

```
cd stm_creator-frontend  
npm install
```

For local backend development, set:

```
VITE_API_BASE_URL=http://localhost:3000
```

Start Vite:

```
npm run dev
```

Open:

```
http://localhost:5173
```

4.4 Production-Like Frontend Build

To point a production build at the deployed backend, set the frontend environment before building:

```
VITE_API_BASE_URL=https://hammerhead-app-t8l9y.ondigitalocean.app
```

Then build:

```
npm run build
```

The deployed editor is:

```
https://stm-8nizc.ondigitalocean.app/editor
```

5. npm Scripts

5.1 Frontend Scripts

Defined in `stm_creator-frontend/package.json`.

Typical scripts:

Script	Purpose
<code>npm run dev</code>	Start Vite dev server.
<code>npm run build</code>	Type-check and build production assets.
<code>npm run preview</code>	Preview built frontend locally.
<code>npm run start</code>	Deployment startup command used by the Procfile.
<code>npm run lint</code>	Run frontend ESLint if configured.

If a script differs, treat `package.json` as the source of truth.

5.2 Backend Scripts

Defined in `tern_backend/package.json`.

Typical scripts:

Script	Purpose
<code>npm run dev</code>	Start backend in development mode.
<code>npm run build</code>	Compile TypeScript into <code>dist/</code> .
<code>npm test</code>	Run Jest tests.
<code>npm run test:unit</code>	Run unit tests where configured.
<code>npm run test:integration</code>	Run integration tests where configured.
<code>npm run lint</code>	Run ESLint.
<code>npm run typecheck</code>	Run TypeScript checks if configured.
<code>npm run migrate:up</code>	Apply database migrations.
<code>npm run migrate:down</code>	Roll back a migration where supported.

Always inspect `package.json` before changing CI or deployment commands.

6. Frontend Code Structure

6.1 Entry and Routes

Important files:

- `src/main.tsx` : React root entry and providers.
- `src/App.tsx` : Main route tree and editor composition.
- `src/pages/Home.tsx` : Home page.

- `src/pages/VerifyEmail.tsx` : Email verification page.
- `src/pages/AdminDashboard.tsx` : Admin dashboard.
- `src/pages/Forbidden.tsx` : Forbidden page.
- `src/pages/NotFound.tsx` : Not-found page.

`App.tsx` is the central integration point for the editor. It wires together graph hooks, toolbar actions, modals, comments, collaboration, and route-level behavior.

6.2 API Layer

Important files:

- `src/app/auth/api.ts`
- `src/app/api/models.ts`
- `src/app/api/comments.ts`
- `src/app/api/drivers.ts`
- `src/app/api/locks.ts`

Guidelines:

- Put REST fetch logic in the API layer, not directly in components.
- Use the shared auth fetch helper so JWT handling stays consistent.
- Keep API response normalization close to the API client.
- Keep UI components focused on rendering and user interaction.

The frontend chooses its API base URL from Vite environment variables. If no local override is present, the source may default to the deployed backend.

6.3 Editor Hook Layer

The graph editor is split across hooks and helper modules:

File	Responsibility
<code>useGraphEditor.ts</code>	Main orchestration hook.
<code>useGraphBaseState.ts</code>	Base editor state.
<code>graphConstants.ts</code>	React Flow node/edge types and defaults.
<code>graphNodes.ts</code>	State node creation, editing, duplication, click behavior.
<code>graphTransitions.ts</code>	Transition creation and transition handlers.
<code>edgeHandlers.ts</code>	Edge clicks, edge changes, transition save behavior.
<code>graphMutations.ts</code>	Local and remote graph patch helpers.
<code>graphRebuilder.ts</code>	Rebuild React Flow edges from model transitions.
<code>graphFilters.ts</code>	Transition filtering.
<code>graphModel.ts</code>	Model loading, saving, refresh, and relay entry points.
<code>graphDeletes.ts</code>	Model, state, and transition delete flows.
<code>graphImportExport.ts</code>	EKS JSON import/export.
<code>graphVersions.ts</code>	Local snapshot versioning.
<code>graphLayout.ts</code>	Graph layout behavior.

When adding editor behavior, first decide which layer owns it:

- UI-only control: component.
- Graph state mutation: hook or graph helper.
- Backend call: API client.
- Data conversion: utility.

6.4 Node and Transition UI

Important files:

- `src/nodes/customNode.tsx`
- `src/nodes/nodeModal.tsx`
- `src/nodes/templates.ts`
- `src/transitions/transitionModal.tsx`
- `src/edges/customEdge.tsx`

State nodes and transition edges are React Flow components. Keep business data in typed state objects and keep visual styling in CSS files.

Transition editing includes driver search, causal-chain fields, preconditions, and delta-related values. Any change to transition data should be checked against backend model types and save service behavior.

6.5 Data Conversion

Important files:

- `src/utils/dataLoader.tsx`
- `src/utils/eksJson.ts`
- `src/utils/stateTransition/types.ts`
- `src/utils/stateTransition/helpers.ts`
- `src/utils/stateTransition/nodes.ts`
- `src/utils/stateTransition/edges.ts`
- `src/utils/stateTransition/layout.ts`

These modules bridge backend STM data and React Flow data.

Development rules:

- Do not duplicate conversion logic inside UI components.
- Keep imported EKS JSON normalization in conversion utilities.
- Preserve stable IDs for states and transitions where possible.
- Keep state positions in the model save path.

6.6 Frontend Permissions

Important files:

- `src/constants/roles.ts`
- `src/utils/permissions.ts`
- `src/components/admin/ProtectedAdminRoute.tsx`
- `src/components/ModelPermissionsModal.tsx`

The frontend should hide or disable actions that the user cannot perform, but the backend remains authoritative. Never rely on frontend-only checks for security.

6.7 Collaboration Client

Important file:

- `src/collab/socket.ts`

The socket client handles:

- JWT-authenticated connection.
- Room join.
- Presence sync.
- Cursor and viewport events.
- Entity lock events.
- Entity patch events.
- Comment and milestone notifications.

Keep socket event names aligned with backend `src/collab/socket.ts`.

7. Backend Code Structure

7.1 Application Startup

Important files:

- `src/index.ts`
- `src/app.ts`
- `src/socket.ts`
- `src/config/env.ts`
- `src/config/database.ts`

Startup flow:

1. Validate environment.
2. Create Express app.
3. Mount middleware.
4. Mount routes.
5. Create HTTP server.
6. Initialize Socket.IO.
7. Register socket authentication and collaboration handlers.
8. Start expired-lock cleanup.
9. Listen on configured port.

7.2 REST Route Layer

Important route files:

File	Mounted Area
<code>routes/auth.ts</code>	<code>/auth</code>
<code>routes/models.ts</code>	<code>/models</code>
<code>routes/permissions.ts</code>	<code>/models/:name/permissions</code>
<code>routes/locks.ts</code>	<code>/models/:name/lock*</code>
<code>routes/drivers.ts</code>	<code>/drivers</code>
<code>routes/collab.ts</code>	<code>/collab</code>
<code>routes/notifications.ts</code>	<code>/notifications</code>
<code>routes/admin.ts</code>	<code>/api/admin</code>

Routes should stay thin. They should parse and validate input, call services, and return responses.

7.3 Middleware

Important middleware:

- `auth.middleware.ts`: JWT bearer auth and verified-email checks.
- `role.middleware.ts`: Global role guard.

- `model-permission.middleware.ts` : Model role guard.
- `requireAdmin.ts` : Admin-only guard.
- `rateLimit.ts` : In-memory rate limiting.
- `requestId.middleware.ts` : Request ID assignment.
- `error.middleware.ts` : Error envelope.
- `validation/validate.ts` : Zod validation wrapper.

Recommended pattern:

```
route -> auth -> role/model permission -> validation -> service -> response
```

7.4 Service Layer

Services hold most business logic.

Important service groups:

- `services/auth.service.ts` : signup, login, verification.
- `services/email.service.ts` : verification email delivery.
- `services/models/show.service.ts` : model reads and aggregation.
- `services/models/save.service.ts` : model save, template clone, owner grant.
- `services/models/remove.service.ts` : delete model/state/transition.
- `services/models/reviewLock.service.ts` : review lock logic.
- `services/permissions/model-permissions.service.ts` : model role grants and lookups.
- `services/drivers/search.service.ts` : driver search.
- `services/collab/*.ts` : activity, comments, locks, mentions, milestones.

Keep database SQL in services or closely owned helpers. Avoid putting SQL in route handlers.

8. Database Design

The schema is managed by migrations under `tern_backend/migrations`.

Important data areas:

- Users and authentication.
- Email verification tokens.
- Contributors.
- STM models.
- States.
- Transitions.
- Drivers and sub-drivers.
- State node positions.
- Model permissions.
- Collaboration locks.
- Comments and mentions.
- Activity log.

- Milestones.
- Admin sessions.
- Audit log.
- Review lock metadata.

Migration order matters. Migrations include initial schema, collaboration tables, transition uniqueness, model permissions, email verification, template flags, admin tables, review lock columns, seeded templates, owner model role, lock type, and cascade/index improvements.

8.1 Common Database Rules

- Use migrations for schema changes.
- Keep forward migrations deterministic.
- Add indexes for new query paths.
- Preserve existing data where possible.
- Use constraints for invariants that must hold across all code paths.
- Update tests when database behavior changes.

8.2 Template Models

Default template names:

- Woodlands
- Grasslands
- Shrublands

Templates should be cloned for user editing. Avoid modifying default templates through normal user editing flows unless the task is explicitly template maintenance.

9. Authentication and Permission Model

9.1 JWT

REST authentication uses:

```
Authorization: Bearer <token>
```

Socket.IO authentication uses token data in the handshake. Both should validate the same user identity model.

Development rules:

- Keep token signing and verification in `utils/jwt.ts`.
- Keep middleware behavior consistent across REST areas.
- Reject unverified users where the route requires verified accounts.
- Do not leak password hash or token secrets in responses.

9.2 Passwords

Passwords are hashed with bcrypt and an application pepper.

Important files:

- `utils/hash.ts`
- `services/auth.service.ts`

Rules:

- Never store plaintext passwords.
- Keep `PASSWORD_PEPPER` out of source control.
- Treat pepper rotation as a migration/operational event.

9.3 Signup and Verification

Signup creates a user and verification token. Verification consumes a token and marks the account verified.

If developing this area:

- Test expired token behavior.
- Test duplicate email behavior.
- Test login before and after verification.
- Test resend verification rate limiting.

9.4 Model Permissions

Global role and model role both matter.

Common model roles:

- `owner`
- `editor`
- `viewer`

Model permission checks should happen on backend routes that read or mutate model-specific resources.

Owner/admin-only operations:

- Manage model permissions.
- Delete model where policy requires ownership.
- Apply high-impact model operations.

10. Real-Time Collaboration

10.1 Backend Collaboration

Important files:

- `src/collab/auth.middleware.ts`
- `src/collab/socket.ts`
- `src/collab/roomManager.ts`
- `src/collab/lockCache.ts`
- `src/collab/lockCleanup.ts`

- `src/collab/types.ts`

Backend responsibilities:

- Authenticate sockets.
- Join users to model rooms.
- Maintain online presence.
- Broadcast cursor and viewport events.
- Acquire, deny, release, and clean up locks.
- Validate and broadcast entity patches.
- Broadcast new comments and milestones.
- Clean locks on disconnect or expiry.

10.2 Frontend Collaboration

Important file:

- `src/collab/socket.ts`

Frontend responsibilities:

- Connect with JWT.
- Join the active model room.
- Show user presence.
- Render or process remote cursors.
- Respect lock state.
- Apply or notify remote patches.
- Cleanly disconnect when leaving a model or logging out.

10.3 Event Compatibility

When changing a socket event:

1. Update backend event handler and emitted payload.
2. Update frontend socket client.
3. Update TypeScript event types where present.
4. Update integration tests.
5. Update API documentation.

Do not rename events casually; this breaks deployed clients.

11. Testing

11.1 Backend Unit and Integration Tests

Backend tests live under:

`tern_backend/tests/`

Coverage includes:

- Auth routes and services.
- Middleware.
- Model routes and services.
- Permission behavior.
- Driver search.
- Collaboration routes and sockets.
- Locks.
- Notifications.
- Admin endpoints.
- Utility functions such as JWT, hashing, and transition delta logic.

Run:

```
cd tern_backend  
npm test
```

If tests depend on a database, make sure the test database and environment variables are configured.

11.2 Postman/Newman

Postman assets:

- `postman/TERN_API.collection.json`
- `postman/local.environment.json`
- `postman/production.environment.json`

Use Newman for repeatable API checks against local or deployed environments.

Before running against production:

- Use non-destructive test data.
- Confirm credentials.
- Confirm environment target URL.

11.3 Frontend Checks

Common frontend checks:

```
cd stm_creator-frontend  
npm run build
```

Run lint if available:

```
npm run lint
```

For UI changes, also manually test:

- Login.

- Model selection.
- Create/edit/delete state.
- Create/edit/delete transition.
- Save model.
- Comments.
- Collaboration if the change touches sockets or locks.
- Permissions if the change touches role logic.
- Import/export if the change touches data conversion.

12. Build and Deployment

12.1 Frontend Build

Frontend build:

```
cd stm_creator-frontend
npm install
npm run build
```

Production environment:

```
VITE_API_BASE_URL=https://hammerhead-app-t8l9y.ondigitalocean.app
```

DigitalOcean frontend route:

```
https://stm-8nizc.ondigitalocean.app/editor
```

The frontend `Procfile` contains:

```
web: npm run start
```

Check the actual `package.json` script for the final runtime command used by the deployment.

12.2 Backend Build

Backend build:

```
cd tern_backend
npm install
npm run build
```

Production backend environment should include:

```
NODE_ENV=production
PORT=<provided-by-platform>
DATABASE_URL=<production-postgres-url>
JWT_SECRET=<secret>
PASSWORD_PEPPER=<secret>
FRONTEND_URL=https://stm-8nizc.ondigitalocean.app
```

Run migrations before serving production traffic:

```
npm run migrate:up
```

12.3 DigitalOcean Deployment Notes

The current deployment is on DigitalOcean.

Operational notes:

- Frontend and backend are separate deployable services.
- Frontend environment variables are baked into the Vite build.
- Changing `VITE_API_BASE_URL` requires a frontend rebuild/redeploy.
- Changing backend CORS or secrets requires backend redeploy.
- Backend `FRONTEND_URL` should match the frontend origin, not necessarily the `/editor` path.

Use:

```
FRONTEND_URL=https://stm-8nizc.ondigitalocean.app
```

The public editor page remains:

```
https://stm-8nizc.ondigitalocean.app/editor
```

12.4 Docker Backend Image

The backend contains a Dockerfile with build and runtime stages.

Typical use:

```
cd tern_backend
docker build -t tern-backend .
```

Before deploying a Docker image, confirm:

- Environment variables are supplied by the platform.
- The runtime command starts the compiled server.
- Migrations are run by a release job, startup script, or a separate migration service.

13. CI

Backend CI lives in:


```
tern_backend/.github/workflows/ci.yml
```

The workflow is intended to cover:

- Install dependencies.
- Lint.
- Type check.
- Build.
- Jest tests.

When adding CI checks:

- Keep them deterministic.
- Avoid depending on local developer paths.
- Use service containers for database-dependent integration tests when needed.
- Keep secrets out of logs.

14. Development Guidelines

14.1 Frontend Guidelines

- Keep API calls in API client modules.
- Keep graph mutations in graph hooks/helpers.
- Keep visual components focused on rendering and user interaction.
- Reuse existing modals and toolbar patterns.
- Keep TypeScript types aligned with backend payloads.
- Test permission-dependent UI with viewer, editor, owner, and admin users.
- Avoid adding global state unless local hook state becomes genuinely unmanageable.

14.2 Backend Guidelines

- Keep routes thin.
- Put SQL and business rules in services.
- Validate request bodies with Zod.
- Use typed errors and the central error middleware.
- Enforce permission on the backend even if the frontend hides controls.
- Add migrations for schema changes.
- Add tests for permission-sensitive and data-shape-sensitive behavior.
- Avoid logging secrets, passwords, JWTs, or verification tokens.

14.3 Documentation Guidelines

When changing behavior, update:

- User-facing operation manual if workflows change.
- Developer manual if architecture, setup, tests, or deployment changes.

- API manual if endpoint, payload, error, auth, or socket events change.
- Swagger/OpenAPI annotations where present.
- Postman collection if request or response shape changes.

15. Common Development Tasks

15.1 Add a REST Endpoint

Recommended steps:

1. Add or update Zod schema under `src/middleware/validation`.
2. Add route handler under the relevant route file.
3. Add service function for business logic and SQL.
4. Apply auth, role, and model permission middleware.
5. Add tests.
6. Update API documentation.
7. Update frontend API client if the frontend consumes the endpoint.

15.2 Add a Database Field

Recommended steps:

1. Create a new migration.
2. Add the column, constraint, or index.
3. Update backend TypeScript types.
4. Update service reads and writes.
5. Update frontend types if exposed to the UI.
6. Update tests.
7. Update import/export conversion if the field belongs to model data.

15.3 Add an Editor Canvas Feature

Recommended steps:

1. Identify whether it is node, transition, edge, layout, toolbar, or import/export behavior.
2. Add UI to the relevant component.
3. Put graph state changes in the relevant hook/helper.
4. Update data conversion if the feature must persist.
5. Update backend save/load types if needed.
6. Test with create, edit, save, reload, and export flows.

15.4 Debug Save Failure

Checklist:

- Inspect browser network response for `POST /models/save`.
- Check HTTP status and error code.
- Confirm JWT exists and is not expired.

- Confirm user email is verified.
- Confirm model role is `editor` or `owner`.
- Confirm review lock is not active.
- Confirm request payload includes required fields.
- Check backend logs.
- Check database constraints.

15.5 Debug Permission Problems

Checklist:

- Confirm global role in the user table.
- Confirm model permission row for the model and user.
- Confirm route middleware order.
- Confirm frontend permission helper logic.
- Confirm API response shape expected by the frontend.

Known source-level caution:

- The model permissions endpoint and frontend permissions modal should agree on the response structure. If one side expects `{ permissions: [...] }` and the other returns a bare array, the UI may show no entries even though the backend has data.

15.6 Debug Collaboration Problems

Checklist:

- Confirm Socket.IO URL and API base URL.
- Confirm JWT is passed in the socket handshake.
- Confirm backend CORS origin includes the frontend origin.
- Confirm the client joins the expected model room.
- Confirm event names match between frontend and backend.
- Confirm locks are released on modal close and socket disconnect.

16. Known Notes and Risks

- The root workspace is documentation-oriented; the app source lives in the two subdirectories.
- Frontend environment variables are build-time values.
- New users are likely `Viewer` by default and may need promotion or model-level grants.
- Default templates should be cloned for editing.
- Permission-sensitive behavior must be enforced by the backend.
- Collaboration has both persistent REST concepts and transient socket lock concepts; do not confuse model review lock with entity editing lock.
- The IDE tab `stm_creator-frontend/FRONTEND_LINT_DEBT.md` was not present on disk during documentation generation.